



VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE204L- Design and Analysis of Algorithms

Dr. Iyappan Perumal

Assistant Professor Senior Grade 2

School of Computer Science & Engineering

VIT,Vellore.



Course Objective

1. To provide mathematical foundations for analysing the complexity of the algorithms
2. To impart the knowledge on various design strategies that can help in solving the real world problems effectively
3. To synthesize efficient algorithms in various engineering design situations



BCSE204L- Design and Analysis of Algorithms-

Course outcome

1. Apply the mathematical tools to analyse and derive the running time of the algorithms
2. Demonstrate the major algorithm design paradigms.
3. Explain major graph algorithms, string matching and geometric algorithms along with their analysis.
4. Articulating Randomized Algorithms.
5. Explain the hardness of real-world problems with respect to algorithmic efficiency and learning to cope with it.



BCSE204L- Design and Analysis of Algorithms- Syllabus

Module:1	Design Paradigms: Greedy, Divide and Conquer Techniques	6 hours
Overview and Importance of Algorithms - Stages of algorithm development: Describing the problem, Identifying a suitable technique, Design of an algorithm, Derive Time Complexity, Proof of Correctness of the algorithm, Illustration of Design Stages - Greedy techniques: Fractional Knapsack Problem, and Huffman coding - <u>Divide and Conquer: Maximum Subarray, Karatsuba faster integer multiplication algorithm.</u>		
Module:2	Design Paradigms: Dynamic Programming, Backtracking and Branch & Bound Techniques	10 hours
Dynamic programming: Assembly Line Scheduling, Matrix Chain Multiplication, Longest Common Subsequence, 0-1 Knapsack, TSP- Backtracking: N-Queens problem, Subset Sum, Graph Coloring- Branch & Bound: LIFO-BB and FIFO BB methods: Job Selection problem, 0-1 Knapsack Problem		
Module:3	String Matching Algorithms	5 hours
Naïve String-matching Algorithms, KMP algorithm, Rabin-Karp Algorithm, Suffix Trees.		
Module:4	Graph Algorithms	6 hours
All pair shortest path: Bellman Ford Algorithm, Floyd-Warshall Algorithm - Network Flows: Flow Networks, Maximum Flows: Ford-Fulkerson, Edmond-Karp, Push Re-label Algorithm – Application of Max Flow to maximum matching problem		
Module:5	Geometric Algorithms	4 hours
Line Segments: Properties, Intersection, sweeping lines - Convex Hull finding algorithms: Graham's Scan, Jarvis' March Algorithm.		
Module:6	Randomized algorithms	5 hours
Randomized quick sort - The hiring problem - Finding the global Minimum Cut.		
Module:7	Classes of Complexity and Approximation Algorithms	7 hours
The Class P - The Class NP - Reducibility and NP-completeness – SAT (Problem Definition and statement), 3SAT, Independent Set, Clique, Approximation Algorithm – Vertex Cover, Set Cover and Travelling salesman		
Module:8	Contemporary Issues	2 hours

BCSE204L- Design and Analysis of Algorithms- References

Text Books:

1. Thomas H. Cormen, C.E. Leiserson, R L. Rivest and C. Stein, Introduction to Algorithms, 2009, 3rd Edition, MIT Press.

Reference Books:

1. Jon Kleinberg, Éva Tardos ,Algorithm Design, Pearson education, 2014.
2. Rajeev Motwani, Prabhakar Raghavan; Randomized Algorithms, Cambridge University Press, 1995 (Online Print – 2013)
3. Ravindra K.Ahuja, Thomas L.Magnanti and James B.Orlin, “ Network Flows: Theory, Algorithms and Applications”, Pearson Education, 2014.



Module I: Design Paradigms: Greedy, Divide and Conquer Techniques(6 Hours)

- **Overview and Importance of Algorithms**
 - **Stages of algorithm development:**
 - Describing the problem
 - Identifying a suitable technique
 - Design of an algorithm
 - Derive Time Complexity,
 - Proof of Correctness of the algorithm
- **Illustration of Design Stages**
 - **Greedy techniques:**
 - Fractional Knapsack Problem
 - Huffman coding
 - **Divide and Conquer:**
 - **Maximum Subarray**
 - **Karatsuba faster integer multiplication algorithm.**



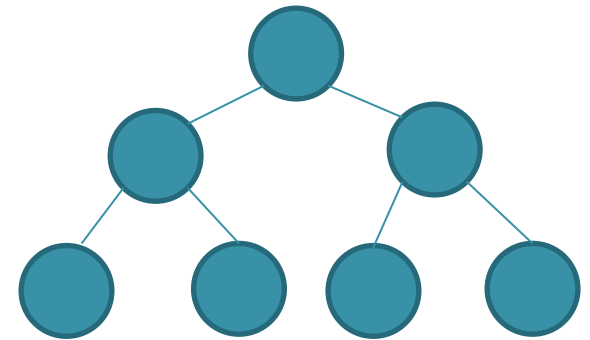
Multiplication of two large n digit numbers

- Multiplication- Multiply two n digit numbers
 - For example: $6734 * 2740$
- How to multiply?
 - Each digit in the second number, we multiply it by all the digits of the first number.
 - **What to be done when we wanted to multiply two large numbers??**
- **Time Complexity : $O(n^2)$ where n is the number of digits.**



KARATSUBA ALGORITHM

- Based on Divide and Conquer
 - Recursively splits the numbers, Performs the required calculations, then combines the results to get the solution of multiplication
 - For Example: $X = 146123$ - 6 digits
 - Split into two parts $A = 146$ & $B = 123$



KARATSUBA ALGORITHM

- $146123 = 146000 + 123$
 $= 146 * 10^3 + 123$

In general it is taken as

$$X = a * 10^{n/2} + b \quad \text{----- Eqn 1}$$

Similarly for another number

$$Y = c * 10^{n/2} + d \quad \text{----- Eqn 2}$$

What is $X * Y = ??$



KARATSUBA ALGORITHM

$$X * Y = (a * 10^{n/2} + b) (c * 10^{n/2} + d)$$

$$= (a * 10^{n/2})(c * 10^{n/2}) + ad * 10^{n/2} + bc * 10^{n/2} + bd$$

$$= \boxed{ac} * 10^{2(n/2)} + \boxed{ad+bc}(10^{n/2}) + \boxed{bd}$$

2 Multiplications-
can be simplified

Takes 1 multiplication,
each, Cant able to
avoid

$$= ad+bc = \boxed{(a+b)(c+d) - ac - bd}$$

Takes 1
Multiplication



KARATSUBA ALGORITHM- DIVIDE AND CONQUER STRATEGY

- **DIVIDE:** Break n-digit numbers into four numbers of $n/2$ each (a,b,c,d).
- **CONQUER:**
 - If $n > 1$: Recursively Compute $ac, bd, (a+b), (c+d)$
 - If $n = 1$: Compute $ac, bd, (a+b), (c+d)$ directly (Base Case)
- **COMBINE:**
 - $10^n * ac + 10^{n/2} ((a+b), (c+d) - ac - bd) + bd$



KARATSUBA ALGORITHM- PSEUDOCODE

$X = \text{Karatsuba}(a, c)$

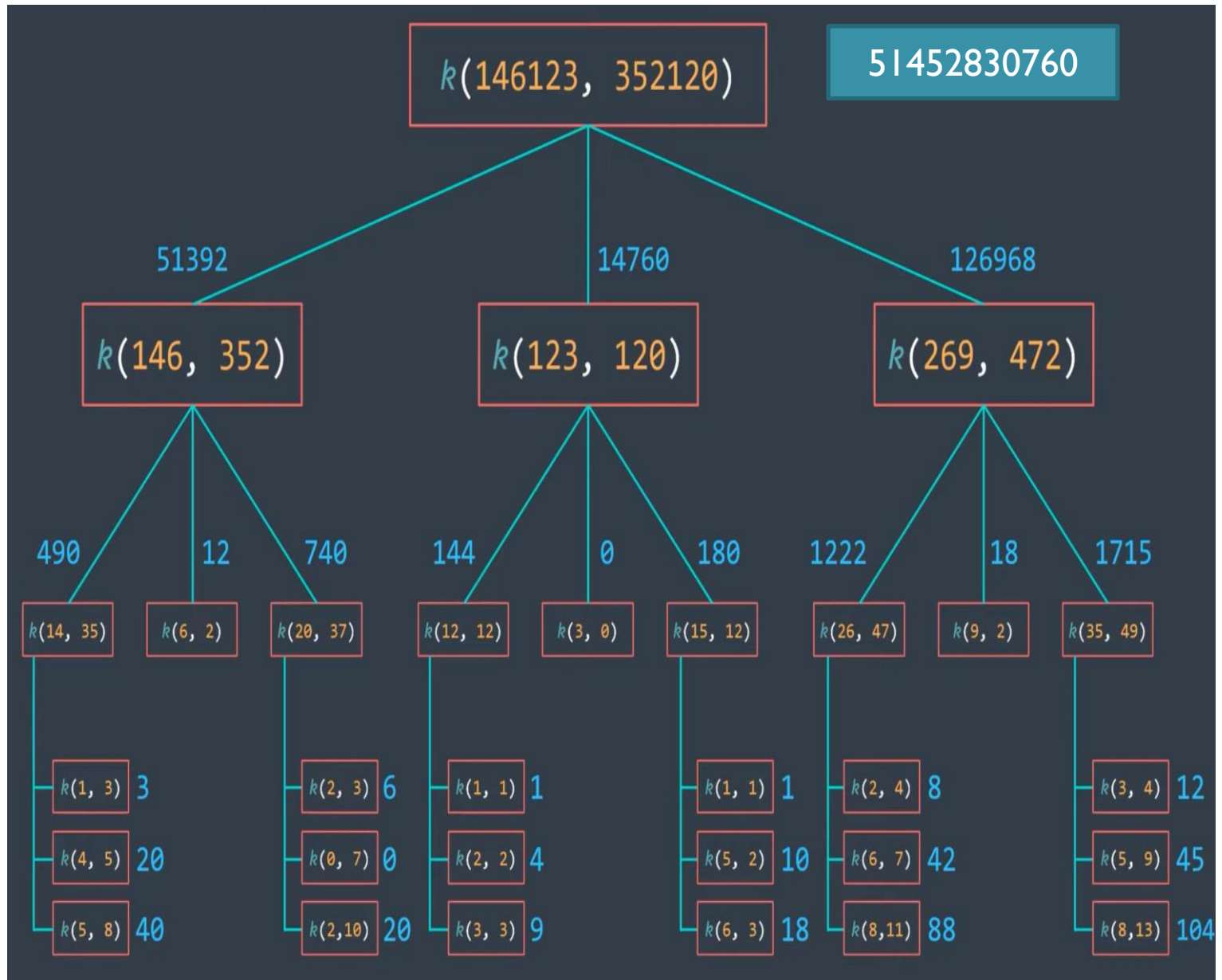
$Y = \text{Karatsuba}(b, d)$

$Z = \text{Karatsuba}(a+b, c+d) - X - Y$

Return $10^n X + 10^{n/2} Z + Y$



KARATSUBA ALGORITHM- Demonstration



KARATSUBA ALGORITHM- Time Complexity

$$T(n) = 3T(n/2) + O(n)$$

- By using Master Theorem

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

- Where $a \geq 1$, $b \geq 1$ and $f(n) = \Theta(n^k \log^p n)$, $k \geq 1$
- Here $a=3$, $b=2$, $k=1$ and $p=0$
- $\log_b a = \log_2 3 = 1.58$ belongs to case 1 ($\log_b a > k$)
- So $T(n) = \Theta(n^{\log_b a}) = \Theta(n^{\log_2 3}) = O(n^{1.58})$
- Which is lesser than normal multiplication time taken

